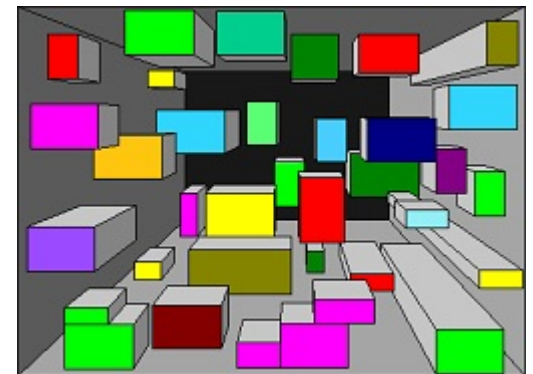


Structuri de Date

Prof. Adina Magda Florea



Curs Nr. 1

- Structuri de date
- Structuri de date abstracte
- ADT colecție și mulțime
- Structuri de date pentru ADT mulțime
- Implementare ADT mulțime
- Noțiuni elementare de analiza algoritmilor

Rezolvarea problemelor

Pentru rezolvarea unei probleme avem nevoie de:

- o reprezentare a universului problemei
- un set de instrumente pentru a transforma această reprezentare
- un criteriu de succes / terminare



Ce este o Structură de date?

O **structură de date** (SD) este un mod de organizare a datelor în memorie pentru accesul eficient la date

Exemple: vector, matrice, structură, listă înlănțuită

```
int a[10];
```

```
struct point {int x; int y;}
```

```
struct key {char *word; int count;}
```

```
struct key keytab[NKEYS];
```

```
struct node {int item; struct node *next;}
```

Ce este o Structură de Date Abstractă?

- O **structură de date abstractă** (SDA), în engleză **Abstract Data Type** (ADT), este o descrie logică a unui mod de organizare a datelor împreună cu operațiile specifice asociate
- Un ADT permite abstractizarea operațiilor fără a ține cont de detalii (de implementare) cât și încapsularea datelor
- Exemple: mulțime, stivă, coadă, graf
 - $\text{push}(\text{stiva}, \text{elem}) \rightarrow \text{stiva}$
 - $\text{empty}(\text{stiva}) \rightarrow 1 / 0$

Colecții și mulțimi

- O **colecție** este un **grup de elemente de același tip** în care pot exista **duplicate**.
- O **mulțime** este o **colecție** ce **nu conține duplicate**.
- O **colecție (respectiv mulțime)** este un ADT care poate fi descris prin operațiile de bază care se pot executa asupra acestei colecții
- **Operațiile de baza** asupra unei colecții pot fi descrise **abstract**, independent de tipul elementelor componente și de modul de reprezentarea în calculator.
- Pentru implementarea acestor operații avem nevoie de reprezentarea datelor în memorie, deci de o **structură de date**

Colecții și mulțimi - ADT

- **Constructori**, care au ca rezultat o colecție nouă sau modifică o colecție existentă
 - `initializare ()` → colecție
 - `adaugare (colecție, element)` → colecție
 - `eliminare (colecție, element)` → colecție
 - `reuniune (colecție, colecție)` → colecție
 - `intersecție (colecție, colecție)` → colecție
 - `diferența (colecție, colecție)` → colecție

Colecții și mulțimi - ADT

- **Operații de caracterizare**, care furnizează informații despre o colecție
 - $\text{vida}(\text{colecție}) \rightarrow 1 / 0$ (da/nu)
 - $\text{cardinal}(\text{colecție}) \rightarrow \text{intreg}$
 - $\text{apartine}(\text{colecție}, \text{element}) \rightarrow 1 / 0$
 - $\text{identice}(\text{colecție}, \text{colecție}) \rightarrow 1 / 0$
 - $\text{include}(\text{colecție}, \text{colecție}) \rightarrow 1 / 0$

Colecții și mulțimi - ADT

- **Semantica** acestor operații abstracte se poate stabili prin scrierea pseudocodului asociat unei operații
 - **adaugare** (colectie, element) → colectie
 colectie ← colectie + element
 intoarce colectie
 - **adaugare** (multime, element) → multime
 daca apartine (multime, element) **atunci**
 intoarce multime
 altfel intoarce multime+ element

Mulțimi/colecții - Implementare

Avem nevoie de o structură de date

- Ce alegem?

- Vector alocat static
- Vector alocat dinamic
- Listă înlănțuită

- Trebuie aleasă o reprezentare care să permită prelucrarea cât mai eficientă a elementelor colecției.
- Dacă operațiile de adăugare / eliminare sunt relativ rare, atunci se pot utiliza vectori de elemente. De obicei se poate stabili o limită superioară a numărului de elemente.

Vectorul elementelor din colecție poate fi alocat:

- static, în cazul în care limita superioară este cunoscută din faza de compilare și este valabilă în marea majoritate a cazurilor
- dinamic, în restul situațiilor

Mulțimi/colecții - Implementare

(a) Vector alocat static

```
#define CAP 100
typedef char Item;
typedef struct
{ Item elem[CAP];
  long dim;
} Multime;
```

```
Multime a;
a.elem[0]='a';
a.dim=1;
```

Mulțimi/colecții - Implementare

(b) Vector alocat dinamic

```
typedef struct
```

```
{ Item *elem;
```

```
    long dim;
```

```
    long cap;
```

```
} Multime;
```

```
Multime b; int k; /* citire k */
```

```
b.elem =(Item *) malloc(k*sizeof(Item));
```

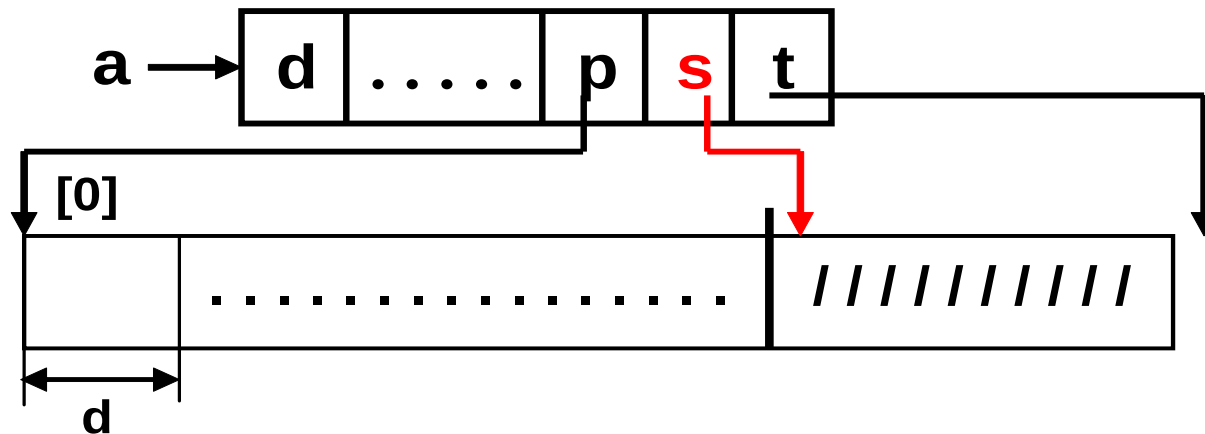
```
b.elem[0]='a';
```

```
b.dim=1;
```

```
b.cap=k;
```

Mulțimi/colecții - Implementare

(c) Vector alocat dinamic, altă variantă

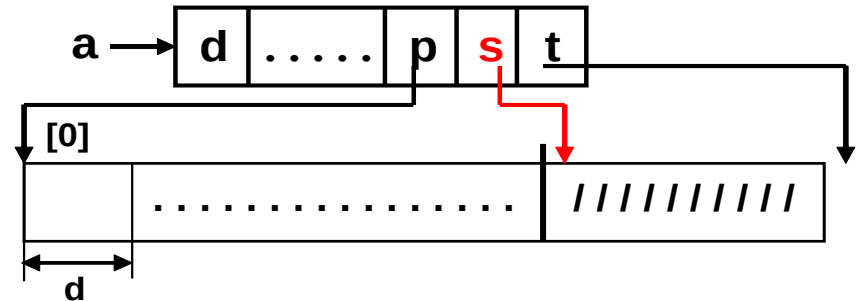


```
typedef struct
{ size_t d;          /* dimensiune elemente */
  .....
  Item *p, *s, *t; /* adresa vector,
                   sfarsit zona utila, alocata */
} Multime;
```

Mulțimi/colecții - Implementare

Vector alocat dinamic, altă variantă

```
typedef struct  
{ size_t d;  
.....  
Item *p, *s, *t} Multime;
```

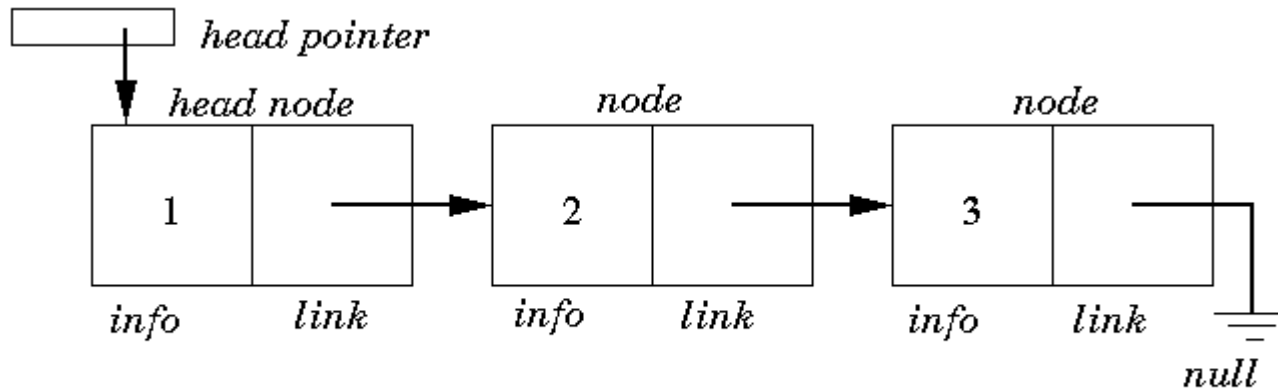


Multime *InitMultime(long n)

```
{ Multime *a; size_t dim_elem;  
  a = (Multime *) malloc(sizeof(Multime));  
  a->d = dim_elem = sizeof(Item);  
  a->p = (Item *) malloc(n*dim_elem);  
  a->s = a->p;  
  a->t = (Item *) (a->p) + n*dim_elem;  
  return a;  
}
```

Mulțimi/colecții - Implementare

(d) Lista înlănțuită – cursul viitor



Căutare în colecții/mulțimi

- Căutarea se realizează pe baza unui **criteriu specific**
- **Metoda de căutare** depinde de modul de memorare a elementelor:
 - **elemente nesortate**
 - căutare **secvențială**, cu oprire la
 - sfârșitul colecției → **esec**
 - îndeplinirea criteriului → **succes**
 - **elemente sortate** conform criteriului de căutare
 - căutare **secvențială**, cu oprire la
 - sfârșitul colecției sau întâlnire succesori → **esec**
 - îndeplinirea criteriului → **succes**
 - căutare **binară**

Căutare în colecții/mulțimi

Algoritm Cautare_Secventiala (cheie) **intoarce**
succes/pozitie sau esec

{ initializari

cat timp mai exista elemente de analizat **repeta**

daca elementul curent = cheie

atunci intoarce succes sau pozitie

altfel avans la urmatorul element;

intoarce esec

}

ADT apartine (colectie, element) $\rightarrow$ 1 / 0

Căutare în colecții/mulțimi

Algoritm Cautare_Secventiala_Sortat (cheie) **intoarce**
succes/pozitie sau esec

{ initializari

cat timp mai exista elemente de analizat **repeta**

daca elementul curent = cheie

atunci intoarce succes sau pozitie

altfel

daca elementul curent > cheie

atunci intoarce esec

altfel avans la urmatorul element;

intoarce esec

}

Căutare în colecții/mulțimi

Algoritm Cautare_Binara_Sortat (cheie) **intoarce**
succes/pozitie sau esec

{ initializari inf, sup

cat timp mai exista elemente de analizat ($\text{inf} \leq \text{sup}$)

repetă

determina m – mijlocul zonei

daca elementul de pe pozitia m = cheie

atunci intoarce succes sau pozitie

altfel

daca elementul de pe pozitia $m <$ cheie

atunci $\text{sup} \leftarrow m$

altfel $\text{inf} \leftarrow m$

intoarce esec

}

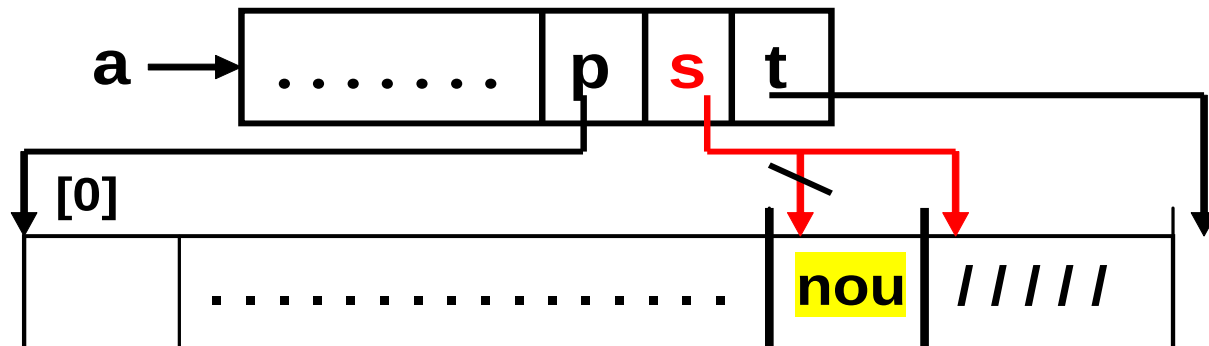
Adăugare element

- Adăugarea unui element se poate realiza numai dacă există **spațiu suficient**

(**a->s** < **a->t**).

- Modul în care se realizează adăugarea depinde de modul de memorare a elementelor:

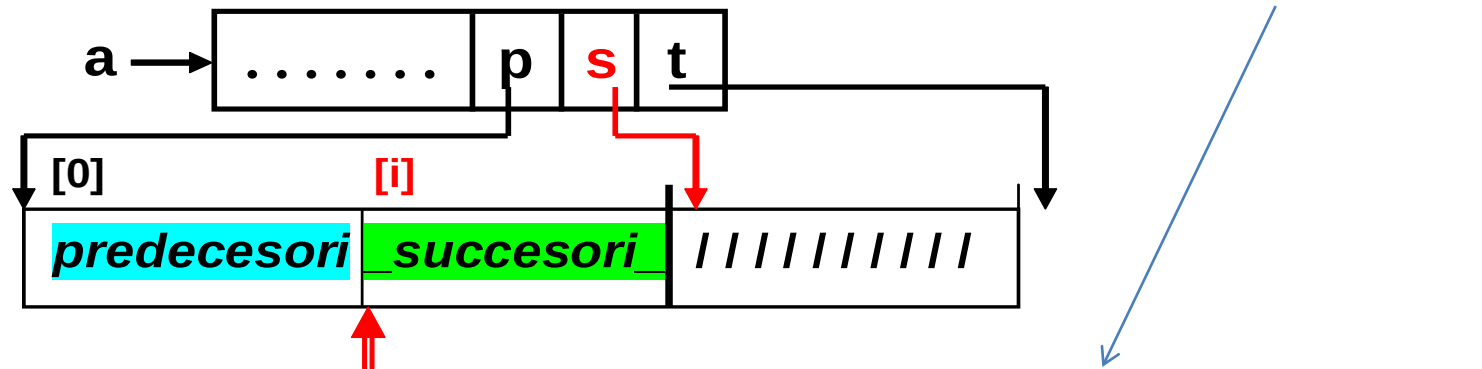
a. elemente nesortate → adăugare la sfârșit



Adăugare element

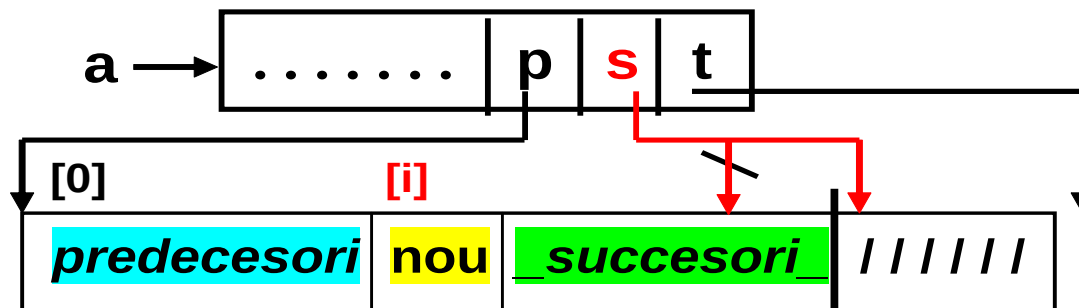
b. **elemente sortate** → inserare în interiorul vectorului, între **predecesori** și **succesori**, realizată în 3 etape:

1. **căutarea** locului de inserare



2. **deplasarea** spre dreapta a succesoriilor

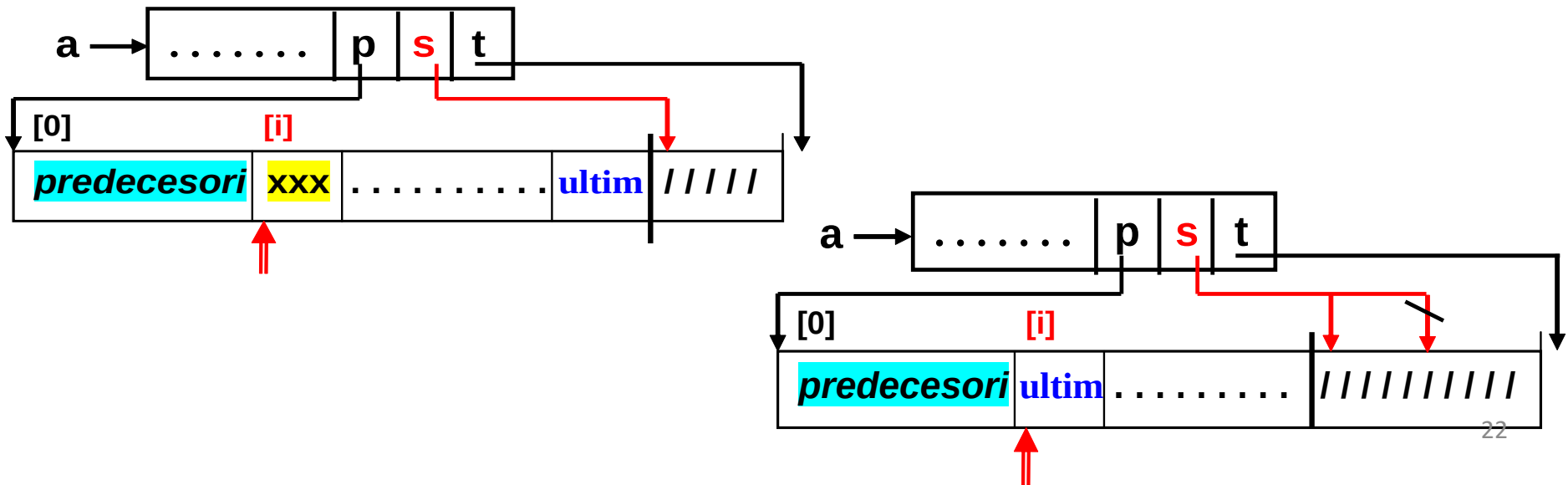
3. **inserarea** propriu-zisă



Eliminare element

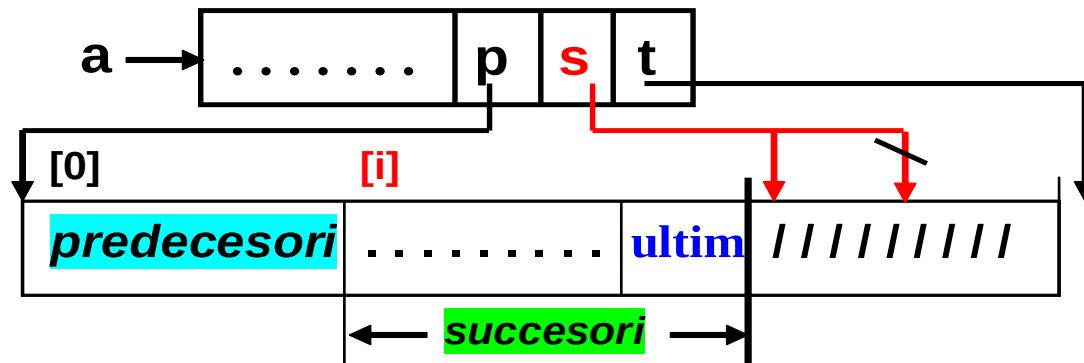
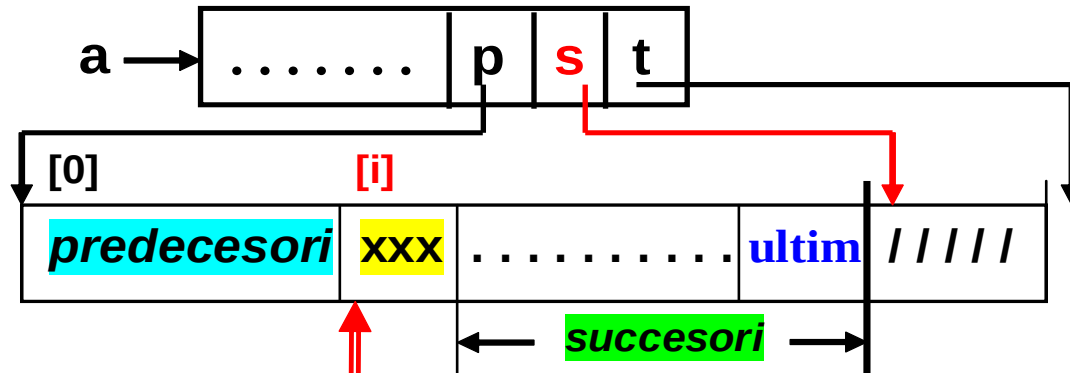
- Eliminarea unui element se realizează în două etape:
 1. **localizare** (căutare)
 2. **eliminare**, dacă a fost găsit
- Modul în care se realizează eliminarea depinde de modul de memorare a elementelor:

a. **elemente nesortate** → **înlocuire prin ultimul** element



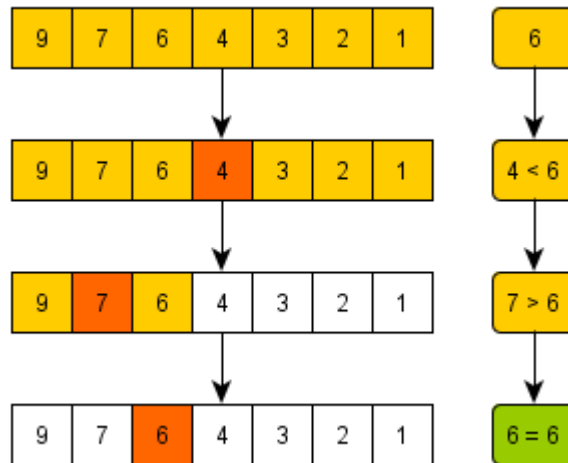
Eliminare element

b. elemente sortate → deplasare spre stânga a succesorilor



Mulțimi sortate

- De ce este mai bine să avem elementele sortate?
- De ce s-a inventat căutarea binară?

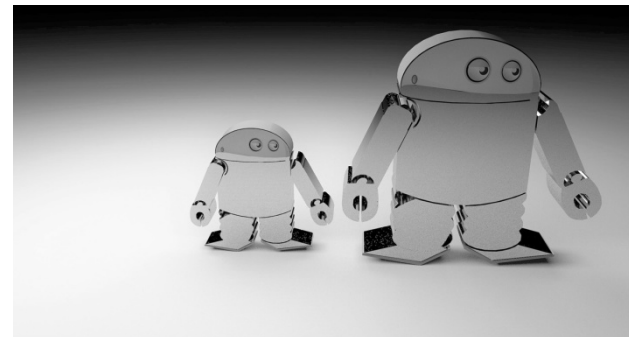


Noțiuni elementare de analiza algoritmilor

- Eficiența algoritmilor
- Considerăm atât operațiile care se execută cât și datele pe care se rulează
- Datele de intrare: cele reale, aleatoare, perverse
- Comparare a 2 algoritmi – analiza algoritmilor
- Identificarea operațiilor abstracte pe care se bazează algoritmul, separând astfel analiza de implementare
- Timpul de rulare – performanța algoritmului și performanța calculatorului

Noțiuni elementare de analiza algoritmilor

- Cazul mediu (average case)
- Cazul cel mai defavorabil (worst case)
- Cazul cel mai defavorabil poate fi rar întâlnit în practică (sau niciodată)
- Ambele ne pot da o indicație asupra performanțelor algoritmilor



Funcții de creștere

- N – un parametru al datelor de intrare care afectează timpul de rulare (**running time**)
- Cei mai mulți algoritmi discutați la curs vor avea timpul de execuție proporțional cu una din următoarele funcții
 1. Cele mai multe instrucțiuni sunt executate o dată sau numai de câteva ori – **timp constant**
 2. **$\log N$** – probleme descompuse în subprobleme – **timp logaritmic**

Funcții de creștere

3. N - O prelucrare se efectuează pe fiecare element de intrare – timp liniar
4. $N \cdot \log N$ – algoritmul rezolvă problema prin descompunere în subprobleme, rezolvă subproblemele și combină apoi intrarea
5. N^2 – algoritmul prelucrează toate perechile formate cu datele de intrare – timp quadratic
6. N^3 - algoritmul prelucrează toate tripletele formate cu datele de intrare – timp cubic
7. 2^N – timp exponențial

Timpul de rulare

Timpul de rulare a unui algoritm va fi de obicei una din aceste funcții înmulțită cu o constantă plus o altă constantă

$N \sim 1$ milion

Op/sec	N	$N \cdot \ln N$	N^2
10^6	sec	sec	sapt
10^9	instant	instant	ore
10^{12}	instant	instant	sec

$N \sim 1$ miliard

Op/sec	N	$N \cdot \ln N$	N^2
10^6	ore	ore	nic
10^9	sec	sec	decade
10^{12}	instant	instant	sapt

Notăția Big-Oh (O mare)

O funcție **$g(N)$** se spune că este de ordinul **$O(f(N))$** dacă există constantele **c_0** și **N_0** astfel încât
 $g(N) < c_0 * f(N)$ pentru **$\forall N > N_0$**

c_0 și **N_0** se referă la detalii de implementare

De obicei dorim un algoritm cu o funcție de creștere cât mai mică

Totuși, în anumite cazuri putem prefera, de ex, un algoritm cu N^2 față de unul cu $\log N$ dacă valorile tipice ale lui N sunt tot timpul sub N_0

Evaluare algoritmi

Algoritm Cautare_Secventiala

```
int search(int a[], int v, int l, int r)
{ int i;
  for (i=l; i<=r; i++)
    if(v==a[i]) return i;
  return -1;
}
```

- Căutarea secvențială examinează N numere pt căutarea fără succes (worst case) și aproximativ N/2 numere pentru căutarea cu succes (average case)
- **$O(N)$**

Evaluare algoritmi

Algoritm Cautare_Binara

```
int search(int a[], int v, int l, int r)
{ while (r >= l)
    { int m = (l+r)/2;
      if (v==a[m]) return m;
      if (v<a[m]) r=m-1;
      else l=m+1;
    }
  return -1;
}
```

- Căutarea binară nu examinează mai mult de $\log N + 1$ numere (worst case)
- **$O(\log N)$**

Evaluare algoritmi

Algoritm Sortare Bubble

```
void Bubble(int a[], int N)
{ int i, j, t;
  for(i = N; i >= 1; i--)
    for(j = 2; j <= i; j++)
      if( a[j-1] > a[j] )
        { t=a[j-1]; a[j-1]=a[j]; a[j]=t; }
}
```

- **$O(N^2)$** cazul mediu și cel mai defavorabil

Evaluare algoritmi

- Algoritm sortare Quicksort - Folosește paradigma **“divide-and-conquer”**
- **Divide:** dacă S (șirul de sortat) are cel puțin două elemente, selectează un element **x** din S – **pivot** (tipic ultimul). Împarte elementele din S în 3 părți:
 - Elemente < decât **x** (L)
 - Elemente = cu **x** (E)
 - Elemente > ca **x** (G)
- **Conquer:** sortează L și G
- **Combine:** Reface S punând în S, în ordine: L, E și G
- **$O(N^2)$** cazul cel mai defavorabil
- **$O(N\log N)$** cazul mediu